

Introduction to Structured Programming KU

Session 4: Advanced Concepts & Data Structures

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

Roadmap: Advanced Concepts

Adv. Dynamic
Memory

2D Arrays
(Static/Dyn)

Recursion

Data Structures
(Lists, BST)

Bit
Operations

Goal

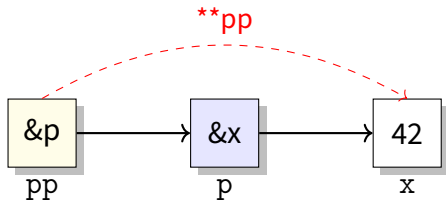
Today we bridge the gap between "Learning C" and "Computer Science". We will build complex structures and manipulate raw bits.

2D Arrays

Review: The Double Pointer (**ptr)

To understand dynamic 2D arrays, we must understand pointers to pointers.

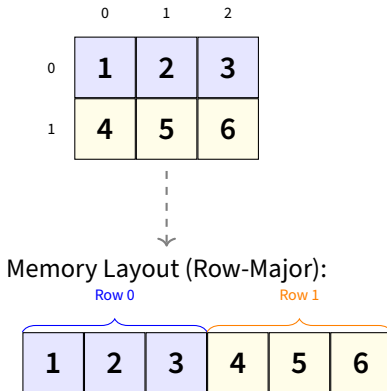
```
1 int x = 42;
2 int *p = &x;
3 int **pp = &p;
4
5 // Dereferencing:
6 // *pp == p (Address of x)
7 // **pp == x (Value 42)
8
```



Static 2D Arrays (Contiguous)

`int matrix[2][3]` is one solid block of memory.

```
1 int m[2][3] =  
2 {  
3     {1, 2, 3},  
4     {4, 5, 6}  
5 };  
6  
7 // Memory is linear!  
8 // 1, 2, 3, 4, 5, 6  
9
```



Accessing Elements

Syntax: `matrix[row_index][col_index]`. Think of it as **Coordinates**: (Row, Column).

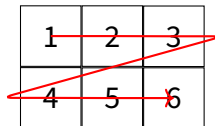
```
1 int m[2][3] =  
2 {  
3     {10, 20, 30},  
4     {40, 50, 60}  
5 };  
6  
7 // Row 1, Col 2  
8 int val = m[1][2];  
9 // val = 60
```

	0	1	2 (Col)
0	10	20	30
1 (Row)	40	50	60

Iterating with Nested Loops

To print the grid, we process it **Row by Row** with a **Nested Loop**.

```
1 // Outer Loop: Rows (i)
2 for (int i = 0; i < 2; i++)
3 {
4     // Inner Loop: Cols (j)
5     for (int j = 0; j < 3; j++)
6     {
7         printf("%d ", m[i][j]);
8     }
9     printf("\n"); // Newline after row
10 }
11
```



1	2	3
4	5	6

Row-Major Order

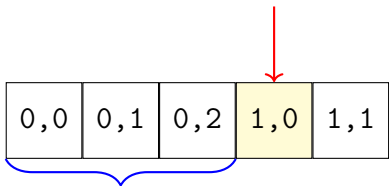
Under the Hood: The Address Formula

The compiler converts $m[i][j]$ into a linear address.

The Formula

$$\text{Address} = \text{Base} + (\text{Row_Index} * \text{Width}) + \text{Col_Index}$$

To get to $m[1][0]$:
Skip 1 full row (3 items)



Row 0 (Width = 3)

Passing 2D Arrays to Functions

The compiler needs to know the **Width** (Column count) to calculate jumps.

✗ Wrong

```
1 // Compiler error!  
2 // How big is a row? I don't know!  
3 void print(int m[][])  
4 {  
5     ...  
6 }  
7
```

✓ Correct

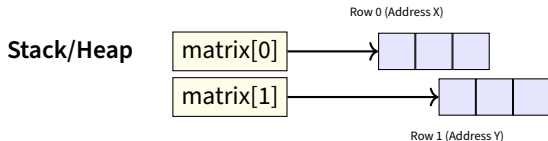
```
1 // OK! Width is fixed to 3.  
2 void print(int m[][3])  
3 {  
4     ...  
5 }  
6
```

Tip: You can omit the first dimension (rows), but never the second (cols)!

Dynamic 2D Arrays (The Loop)

We cannot just `malloc(rows * cols)`. We usually want an **Array of Pointers**.

```
1 // 1. Allocate array of ROW POINTERS
2 int **matrix = malloc(rows * sizeof(int*));
3
4 // 2. Allocate each ROW individually
5 for (int i = 0; i < rows; i++)
6 {
7     matrix[i] = malloc(cols * sizeof(int));
8 }
9
10 matrix[1][2] = 99; // Access works the same!
11
```



Freeing Dynamic 2D Arrays

Order Matters!

You must free the **inside** (rows) before you free the **outside** (backbone).

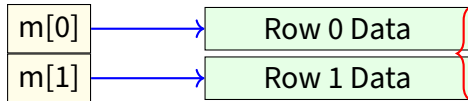
```
1 // 1. Free Rows
2 for (int i = 0; i < rows; i++)
3 {
4     free(matrix[i]);
5 }
6
7 // 2. Free Backbone
8 free(matrix);
9
```

Optimization: The Contiguous Block

Problem: Calling `malloc` in a loop fragments memory (slow).

Solution: Allocate **one giant block** for data, and point rows into it.

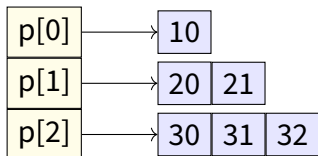
```
1 int **matrix = malloc(rows * sizeof(int*));
2
3 // 1. One giant alloc for ALL integers
4 int *flat = malloc(rows * cols * sizeof(int));
5
6 // 2. Point row pointers into the flat block
7 for (int i = 0; i < rows; i++)
8 {
9     // Jump 'cols' spots for each row
10    matrix[i] = flat + (i * cols);
11 }
12
13 // Freeing is easy!
14 free(matrix[0]); // Frees the big block
15 free(matrix);    // Frees the spine
16
```



Feature: Jagged (Ragged) Arrays

Since each row is a separate allocation, they can have **different lengths!**

```
1 int **triangle =
2     malloc(3 * sizeof(int*));
3
4 for (int i = 0; i < 3; i++)
5 {
6     // Row 0 -> size 1
7     // Row 1 -> size 2
8     // Row 2 -> size 3
9     triangle[i] =
10    malloc((i + 1) * sizeof(int));
11 }
12
```



Safety: Handling Allocation Failure

What if `malloc` fails on Row 5? You must free Rows 0-4 before returning!

```
1 for (int i = 0; i < rows; i++)
2 {
3     matrix[i] = malloc(cols * sizeof(int));
4     if (matrix[i] == NULL)
5     {
6         // EMERGENCY CLEANUP!
7         for (int k = 0; k < i; k++)
8         {
9             free(matrix[k]); // Free previous rows
10        }
11        free(matrix); // Free backbone
12        return NULL;
13    }
14 }
```

Warning: `int **` is NOT `int [] []`

They look the same when used (`m[0][0]`), but the memory layout is totally different.

Dynamic (`int **`)

- A pointer to pointers.
- Requires **two** memory lookups.
- `Addr -> Ptr -> Value`

Static (`int m[2][3]`)

- A single flat block.
- Requires **one** math calculation.
- `Base + (row*w + col)`

Compilation Error

```
void func(int **m)
```

You **cannot** pass a static array `int mat[3][3]` to this function! The types are incompatible.

Passing Static 2D Arrays

When passing a static array (e.g., `int m[5][10]`), you **must** specify the column width in the argument.

Function Signature:

```
1 // Rows can be empty []
2 // Cols MUST be specified [3]
3 void printStatic(int m[][3], int rows)
4 {
5     for (int i = 0; i < rows; i++)
6         printf("%d ", m[i][0]);
7 }
8
9 int main(void) {
10     int mat[2][3] = {{1,2,3},{4,5,6}};
11     printStatic(mat, 2);
12 }
13
```

Why?

The compiler needs the width to calculate address jumps:

Addr =

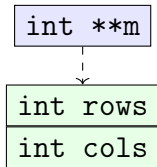
Base + (Row * 3) + Col



Passing Dynamic 2D Arrays

Dynamic arrays are just pointers to pointers. The function knows **nothing** about dimensions, so you must pass them explicitly.

```
1 // Syntax: int **m
2 void printDyn(int **m, int rows, int cols)
3 {
4     for (int i = 0; i < rows; i++) {
5         for (int j = 0; j < cols; j++) {
6             // Two lookups:
7             // 1. Get Row Ptr (m[i])
8             // 2. Get Value ( [j] )
9             printf("%d ", m[i][j]);
10        }
11    }
12 }
```



Required for
loops!

Warning: Types are NOT Interchangeable

You cannot pass a Static Array to a function expecting a Dynamic Array (or vice versa).

Static Array (`int m[3][3]`)

Flat Memory Block

Dynamic Array (`int **m`)

Spine

Row

```
1 printDyn(m, 3, 3);  
2 // ERROR: expected 'int **'  
3 // but argument is 'int (*)[3]'  
4
```

```
1 printStatic(m, 3);  
2 // ERROR: expected 'int (*)[3]'  
3 // but argument is 'int **'  
4
```

Takeaway

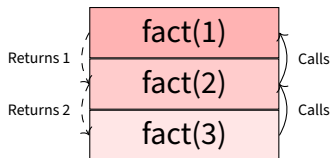
Choose **one** strategy (Static or Dynamic) and stick to it for the entire lifecycle of that variable.

Recursion

Recursion: The Stack Visualization

Every function call creates a new Stack Frame. `fact(3)` calls `fact(2)`, which calls `fact(1)`...

```
1 int fact(int n)
2 {
3     if (n == 1)
4     {
5         return 1;
6     }
7     return n * fact(n-1);
8 }
9
```



The Two Golden Rules

Every recursive function **must** have two parts:

- 1 **Base Case:** The condition to STOP.
- 2 **Recursive Step:** The move towards the base case.

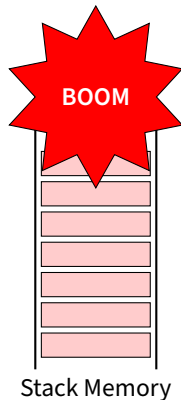
```
1 int sum(int k)
2 {
3     // 1. Base Case
4     if (k <= 0) return 0;
5
6     // 2. Recursive Step
7     return k + sum(k - 1);
8 }
9
```

⚠ Warning
Without Rule #1,
the program runs forever
(until it crashes).

What is a "Stack Overflow"?

The Stack has a fixed size (usually a few MBs). If you recurse too deep (or forever), you run out of space.

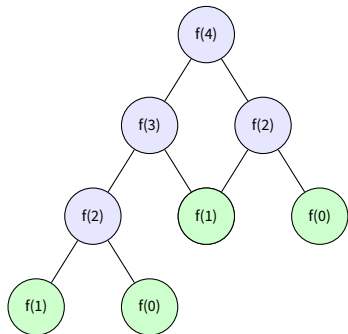
```
1 void forever()  
2 {  
3     // No Base Case!  
4     forever();  
5 }  
6  
7 int main(void)  
8 {  
9     forever();  
10 }  
11
```



Segfault!
(Stack Limit)

The Cost of Recursion (Fibonacci)

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$. Calculating $\text{fib}(4)$ creates a tree of calls.



Notice:

$f(2)$ is computed **twice**.

$f(1)$ is computed **3 times**.

Complexity: $O(2^n)$ (Bad!)

Recursion: Order Matters

The position of your code relative to the recursive call determines the order of execution.

1. Count Down (Head)

```
1 void down(int n)
2 {
3     if (n == 0)
4         return;
5     printf("%d ", n); // Action FIRST
6     down(n - 1);      // Recurse
7 }
8 // Output: 3 2 1
9
```

Prints while diving deep

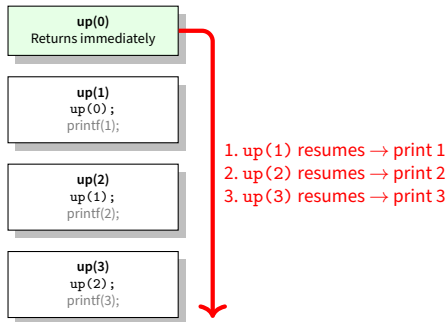
2. Count Up (Tail/Unwind)

```
1 void up(int n)
2 {
3     if (n == 0)
4         return;
5     up(n - 1);        // Recurse FIRST
6     printf("%d ", n); // Action LATER
7 }
8 // Output: 1 2 3
9
```

Prints while returning

Visualizing the Unwind

When `up(n-1)` is called, the current function **pauses** at that line. It only resumes when the child returns.

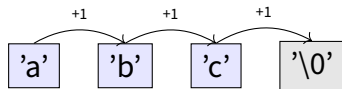


Thinking Recursively: String Length

Problem: Calculate length of string "abc".

Recursive Thought: Length is $1 + \text{length}(\text{"bc"})$.

```
1 int len(char *s)
2 {
3     // Base Case: End of string
4     if (*s == '\0')
5         return 0;
6
7     // Recursive Step:
8     // 1 (me) + len(neighbor)
9     return 1 + len(s + 1);
10 }
11
```



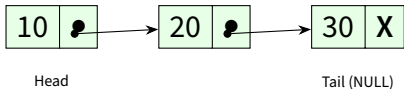
Returns 0

Linked Lists

Linked Lists: The Train

Arrays are fixed size. Linked Lists grow dynamically. Each "Node" holds **Data** and a **Pointer** to the next node.

```
1 struct Node
2 {
3     int data;
4     struct Node *next;
5 };
6
```



Comparison: Arrays vs. Linked Lists

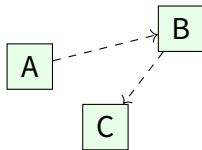
Array (Static/Contiguous)

- Fast Access ($\mathcal{O}(1)$)
- Fixed Size (Resize is slow)
- Needs contiguous RAM.



Linked List (Dynamic)

- Slow Access ($\mathcal{O}(n)$)
- Infinite Size (Limit is RAM)
- Uses scattered memory.

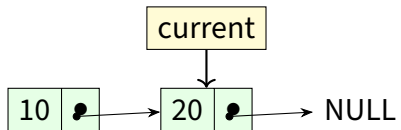


Traversal: The "Hopping" Loop

You cannot jump to index i . You must follow the pointers starting from head.

```
1 void print_list(struct Node *head)
2 {
3     struct Node *current = head;
4     // Loop until we hit NULL
5     while (current != NULL)
6     {
7         printf("%d -> ", current->data);
8         // Move to the next node
9         current = current->next;
10    }
11    printf("NULL\n");
12 }
13
```

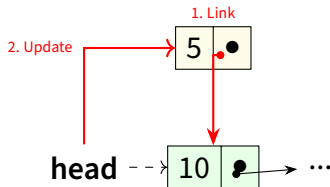
Traversal: The "Hopping" Loop



Insertion: Push Front (Add to Head)

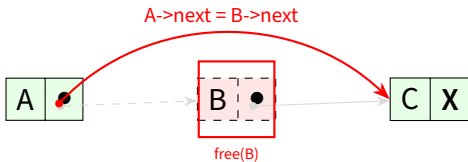
Goal: Add "5" to the front. **Rule:** Always set the *new node's* pointer **before** changing head.

```
1 struct Node *new_node = malloc(...);  
2 new_node->data = 5;  
3  
4 // 1. Link New -> Old Head  
5 new_node->next = head;  
6  
7 // 2. Update Head  
8 head = new_node;  
9
```



Deletion: Bridging the Gap

To remove node **B**, we simply make **A** point to **C**. Then we `free(B)`.

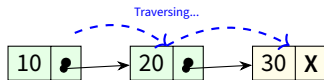


Insertion: Push Back (Append)

To add to the end, you must first **find the last node**.

```
1 void append(struct Node **head_ref, int new_data) {
2     struct Node *new_node = malloc(sizeof(struct Node));
3     new_node->data = new_data;
4     new_node->next = NULL;
5
6     // Case 1: List is empty
7     if (*head_ref == NULL) {
8         *head_ref = new_node;
9         return;
10    }
11
12    // Case 2: Traverse to end
13    struct Node *last = *head_ref;
14    while (last->next != NULL) {
15        last = last->next;
16    }
17    last->next = new_node; // Link!
18 }
19
```

Insertion: Push Back (Append)

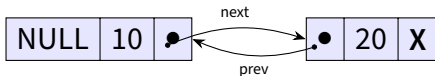


Doubly Linked Lists

Nodes have pointers to both the **Next** and **Previous** node.

- 👍 Can traverse backwards.
- 🗣️ Twice the memory overhead for pointers.

```
1 struct Node {  
2     int data;  
3     struct Node *next;  
4     struct Node *prev; // New!  
5 };  
6
```



Best Practice: The "Constructor" Helper

Don't write malloc every time. Write a helper function!

```
1 struct Node* create_node(int value)
2 {
3     // 1. Allocate
4     struct Node* n = malloc(sizeof(struct Node));
5
6     // 2. Check Memory
7     if (!n)
8     {
9         printf("Memory Error");
10        return 1;
11    }
12
13    // 3. Initialize
14    n->data = value;
15    n->next = NULL; // Crucial!
16
17    return n;
18 }
19
```

Usage:

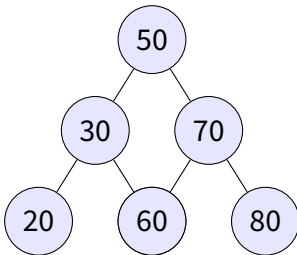
```
struct Node *head = create_node(10);
head->next = create_node(20);
```

- ✓ Safer code
- ✓ Less typing

Binary Search Tree

Binary Search Trees (BST)

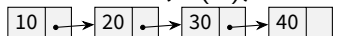
Organized for fast searching. **Rule:** Left Child $<$ Node $<$ Right Child.



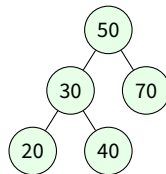
Finding **40**? Start at 50 $\rightarrow$ Go Left (to 30) $\rightarrow$ Go Right (found 40).

Why BST? Efficiency Matter

Linked List ($\mathcal{O}(N)$)



Must check every node!



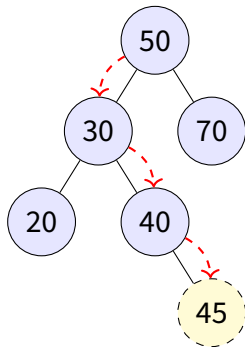
Halves the problem
at every step.

BST ($\mathcal{O}(\log N)$)

Insertion: Where does it go?

Task: Insert 45. Logic:

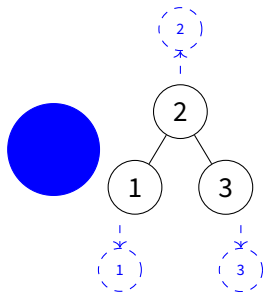
- 1 Start at Root (50). $45 < 50 \rightarrow$ Go Left.
- 2 At Node (30). $45 > 30 \rightarrow$ Go Right.
- 3 At Node (40). $45 > 40 \rightarrow$ Go Right.
- 4 Found NULL? Place 45 there.



Traversal: Printing Sorted Data

In-Order Traversal: Left $\rightarrow$ Self $\rightarrow$ Right. This recursively prints the tree in ascending order!

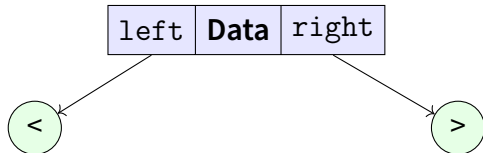
```
1 void print(struct Node *n)
2 {
3     if (n == NULL)
4         return;
5
6     print(n->left); // 1. Left
7
8     printf("%d ", n->data); // 2. Self
9
10    print(n->right); // 3. Right
11 }
```



BST Implementation: The Struct

A Tree Node is just a container with data and two pointers to its children.

```
1 struct Node
2 {
3     int data;
4
5     // The Left Child
6     // (Smaller values)
7     struct Node *left;
8
9     // The Right Child
10    // (Larger values)
11    struct Node *right;
12 };
13
```



BST Implementation: Creating Nodes

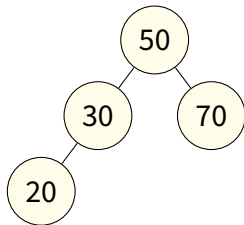
Always initialize pointers to NULL to mark the bottom of the tree.

```
1 struct Node* create_node(int value)
2 {
3     struct Node* n = malloc(sizeof(struct Node));
4     // Check for memory error!
5     if (!n)
6         return NULL;
7     n->data = value;
8     // CRITICAL: Leaf nodes have no children
9     n->left = NULL;
10    n->right = NULL;
11    return n;
12 }
13
```

Building a Simple Tree



```
1 int main(void)
2 {
3     // 1. Create Root
4     struct Node *root = create_node(50);
5
6     // 2. Attach Children
7     root->left = create_node(30);
8     root->right = create_node(70);
9
10    // 3. Go deeper
11    root->left->left = create_node(20);
12 }
13
```



Bitwise Operations

Bitwise Operators

Manipulate variables at the bit level (0 and 1).

- $\&$ (AND): Both 1 $\rightarrow$ 1
- $|$ (OR): One 1 $\rightarrow$ 1
- $\wedge$ (XOR): Different $\rightarrow$ 1
- $\sim$ (NOT): Flip all
- $\ll / \gg$: Shift

Example: 5 & 3

5:	0	1	0	1
3:	0	0	1	1
&:	0	0	0	1

Bitwise Logic: AND (&) vs OR (|)

AND (&)

Both must be 1.

5	0	1	0	1
3	0	0	1	1
1	0	0	0	1

OR (|)

Either can be 1.

5	0	1	0	1
3	0	0	1	1
7	0	1	1	1

Bitwise Logic: XOR vs NOT



XOR (^)

1 if Different.

5	0	1	0	1
3	0	0	1	1
6	0	1	1	0

NOT (~)

Flip every bit.

5	0	1	0	1
				
10	1	0	1	0

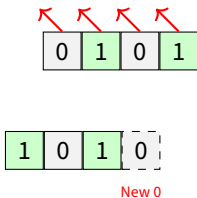
*(Assuming 4-bit)

Bit Shifting: Left ($\ll$) & Right ($\gg$)

Shifting moves bits sideways. Gaps are filled with 0.

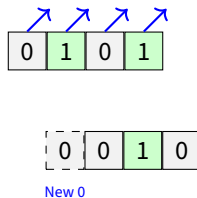
Left Shift ($\ll 1$)

Multiply by 2



Right Shift ($\gg 1$)

Divide by 2



Bit Masking (Flags)

Using bits as boolean flags (very common in embedded/systems).

```
1 #define READ  (1 << 0) // 0001
2 #define WRITE (1 << 1) // 0010
3 #define EXEC  (1 << 2) // 0100
4
5 unsigned char permissions = 0;
6
7 // Set Flag (OR)
8 permissions |= READ;
9
10 // Check Flag (AND)
11 if (permissions & READ) { ... }
12
13 // Clear Flag (AND NOT)
14 permissions &= ~READ;
15
```

Security: Constant-Time Comparison

Problem: Standard `strcmp` returns early (Timing Attack risk!).

Solution: Accumulate differences using `|=` and `^`.

```
1 int secure_compare(char *s1, char *s2)
2 {
3     unsigned char diff = 0;
4
5     // Assume same length for simplicity
6     for (int i = 0; i < 16; i++)
7     {
8         // XOR: 0 if same, 1 if diff
9         // OR: Accumulates the "1"s
10        diff |= (s1[i] ^ s2[i]);
11    }
12
13    // If diff is still 0, strings match!
14    return (diff == 0);
15 }
16
```

Standard strcmp

'P'=='P'? Yes.

'a'=='a'? Yes.

's'=='X'? **STOP!**

Secure Compare

'P'=='P'? `diff |= 0`

'a'=='a'? `diff |= 0`

's'=='X'? `diff |= 1`

... keep going ...

Result: `diff` is 1

Optimization: Even or Odd?

Fact: An integer is odd if its Least Significant Bit (LSB) is 1.

```
1 // Slow (Division)
2 if (x % 2 != 0)
3 {
4     printf("Odd");
5 }
6
7 // Fast (Bitwise AND)
8 if (x & 1)
9 {
10    printf("Odd");
11 }
12
```

13 = 110¹_↓
LSB

The red bit decides!

The Bitwise Toolkit: Flags

```
1 #define ENABLED    (1 << 0) // 0001
2 #define VISIBLE    (1 << 1) // 0010
3 #define BROKEN      (1 << 2) // 0100
4
5 unsigned char flags = 0;
6
7 // 1. SET (Turn ON) -> OR
8 flags |= ENABLED;
9
10 // 2. CLEAR (Turn OFF) -> AND NOT
11 flags &= ~VISIBLE;
12
13 // 3. TOGGLE (Flip) -> XOR
14 flags ^= BROKEN;
15
16 // 4. CHECK (Is it ON?) -> AND
17 if (flags & ENABLED) { ... }
18
```

Final Remarks

Foundations of C Programming

In this course you did (hopefully) learn the fundamentals:

- 💡 **Think** like a programmer
- 🧩 **Divide and Conquer** complex problems
- 🐛 **Test and Debug** your code
- ⚙️ Understand how a program **works**



Questions?

Good luck with the final exercises!

Introduction to Structured Programming KU

Session 4: Advanced Concepts & Data Structures

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security